Deep Reinforcement Learning

Unit 7: Multi-Agent Reinforcement Learning

Hugging Face Deep RL Course

Complete MDX-to-LaTeX study edition

Source: Hugging Face Deep RL Course (original English text).

Contents

1 Introduction **2**
- 1.1 Course Maintenance Notice [Maintenance notice] . . . 3

2 An introduction to Multi-Agents Reinforcement Learning (MARL) **3**
- 2.1 From single agent to multiple agents . . . 3
- 2.2 Different types of multi-agent environments . . . 5

3 Designing Multi-Agents systems **6**
- 3.1 Decentralized system . . . 6
- 3.2 Centralized approach . . . 7

4 Self-Play: a classic technique to train competitive agents in adversarial games **7**
- 4.1 What is Self-Play? . . . 8
- 4.2 Self-Play in MLAgents . . . 9
- 4.3 The ELO Score to evaluate our agent . . . 9

5 Hands-on **11**
- 5.1 What is AI vs. AI? . . . 12
- 5.2 Step 0: Install MLAgents and download the correct executable . . . 12
- 5.3 Step 1: Understand the environment . . . 13
- 5.4 Step 2: Understand MA-POCA . . . 15
- 5.5 Step 3: Define the config file . . . 16
- 5.6 Step 4: Start the training . . . 17
- 5.7 Step 5: **Push the agent to the Hugging Face Hub** . . . 17
- 5.8 Step 6: Verify that your model is ready for AI vs AI Challenge . . . 19
- 5.9 Step 7: Visualize some match in our demo . . . 20

6 Quiz **20**
- 6.1 Q1: Chose the option which fits better when comparing different types of multi-agent environments 20
- 6.2 Q2: Which of the following statements are true about `decentralized` learning? . . . 20
- 6.3 Q3: Which of the following statements are true about `centralized` learning? . . . 20
- 6.4 Q4: Explain in your own words what is the `Self-Play` approach . . . 20
- 6.5 Q5: When configuring `Self-play`, several parameters are important. Could you identify, by their definition, which parameter are we talking about? . . . 20
- 6.6 Q6: What are the main motivations to use a ELO rating Score? . . . 21
- 6.7 Quiz solutions and explanations . . . 21

7 Conclusion **22**
- 7.1 Keep Learning, Stay awesome . . . 22

8 Additional Readings **22**
- 8.1 An introduction to multi-agents . . . 22
- 8.2 Self-Play and MA-POCA . . . 22

1 Introduction



Figure 1: Hugging Face Deep Reinforcement Learning Course cover [original figure]

Since the beginning of this course, we learned to train agents in a *single-agent system* where our agent was alone in its environment: it was **not cooperating or collaborating with other agents**.

This worked great, and the single-agent system is useful for many applications.



Figure 2: A patchwork of all the environments you've trained your agents on since the beginning of the course [original figure]

But, as humans, **we live in a multi-agent world**. Our intelligence comes from interaction with other agents. And so, our **goal is to create agents that can interact with other humans and other agents**.

Consequently, we must study how to train deep reinforcement learning agents in a *multi-agents system* to build robust agents that can adapt, collaborate, or compete.

So today we're going to **learn the basics of the fascinating topic of multi-agents reinforcement learning (MARL)**.

And the most exciting part is that, during this unit, you're going to train your first agents in a multi-agents system: **a 2vs2 soccer team that needs to beat the opponent team**.

1.1 Course Maintenance Notice [Maintenance notice]

Please note that this **Deep Reinforcement Learning course is now in a low-maintenance state**. However, it **remains an excellent resource to learn both the theory and practical aspects of Deep Reinforcement Learning**.

Keep in mind the following points:

- *Unit 7 (AI vs AI)* : This feature is currently non-functional. However, you can still train your agent to play soccer and observe its performance. But the leaderboard for AI vs AI soccer was shut down.

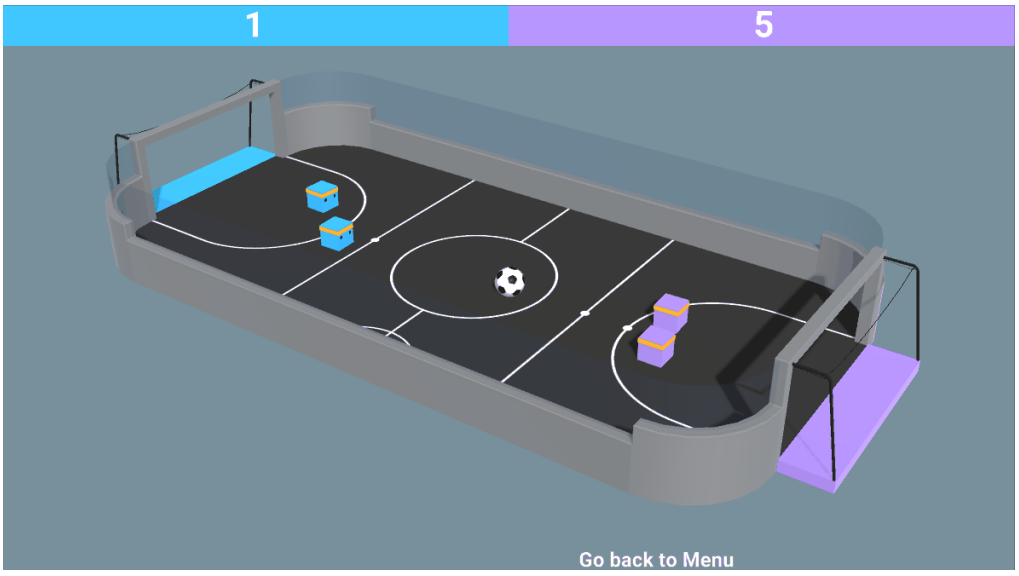


Figure 3: SoccerTwos: Unity ML-Agents two-versus-two soccer environment (Unity ML-Agents Team) [original animation]

So let's get started!

2 An introduction to Multi-Agents Reinforcement Learning (MARL)

2.1 From single agent to multiple agents

In the first unit, we learned to train agents in a single-agent system. When our agent was alone in its environment: **it was not cooperating or collaborating with other agents**.



Figure 4: A patchwork of all the environments you've trained your agents on since the beginning of the course [original figure]

When we do multi-agents reinforcement learning (MARL), we are in a situation where we have multiple agents **that share and interact in a common environment**.

For instance, you can think of a warehouse where **multiple robots need to navigate to load and unload packages**.



Figure 5: Autonomous warehouse robots, illustration by upklyak (Freepik). [original figure]

Or a road with **several autonomous vehicles**.

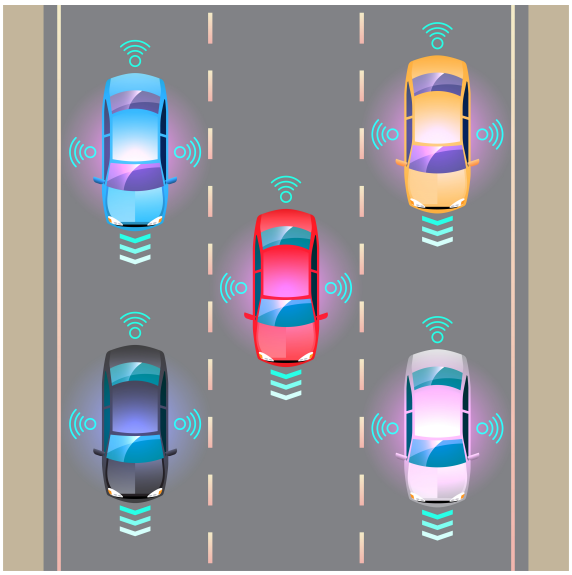


Figure 6: Autonomous vehicles interacting in traffic, illustration by jcomp (Freepik). [original figure]

In these examples, we have **multiple agents interacting in the environment and with the other agents**. This implies defining a multi-agents system. But first, let's understand the different types of multi-agent environments.

2.2 Different types of multi-agent environments

Given that, in a multi-agent system, agents interact with other agents, we can have different types of environments:

- *Cooperative environments*: where your agents need **to maximize the common benefits**.

For instance, in a warehouse, **robots must collaborate to load and unload the packages efficiently (as fast as possible)**.

- *Competitive/Adversarial environments*: in this case, your agent **wants to maximize its benefits by minimizing the opponent's**.

For example, in a game of tennis, **each agent wants to beat the other agent**.

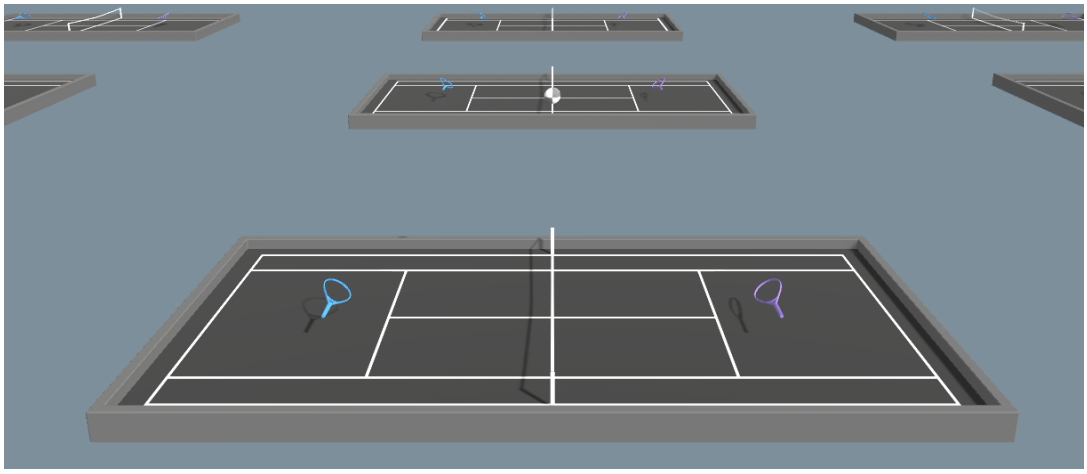


Figure 7: Two agents competing in a game of tennis [original figure]

- *Mixed of both adversarial and cooperative*: like in our SoccerTwos environment, two agents are part of a team (blue or purple): they need to cooperate with each other and beat the opponent team.

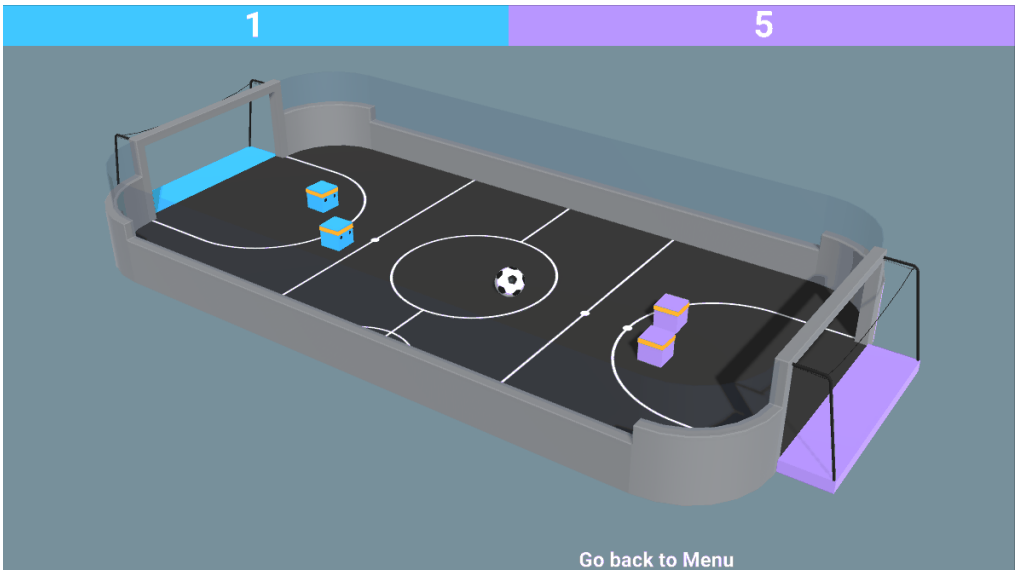


Figure 8: SoccerTwos: Unity ML-Agents two-versus-two soccer environment (Unity ML-Agents Team) [original animation]

So now we might wonder: how can we design these multi-agent systems? Said differently, **how can we train agents in a multi-agent setting** ?

3 Designing Multi-Agents systems

For this section, you're going to watch this excellent introduction to multi-agents made by Brian Douglas.

Watch the original multi-agent system design lecture on YouTube

In this video, Brian talked about how to design multi-agent systems. He specifically took a multi-agents system of vacuum cleaners and asked: **how can can cooperate with each other**?

We have two solutions to design this multi-agent reinforcement learning system (MARL).

3.1 Decentralized system

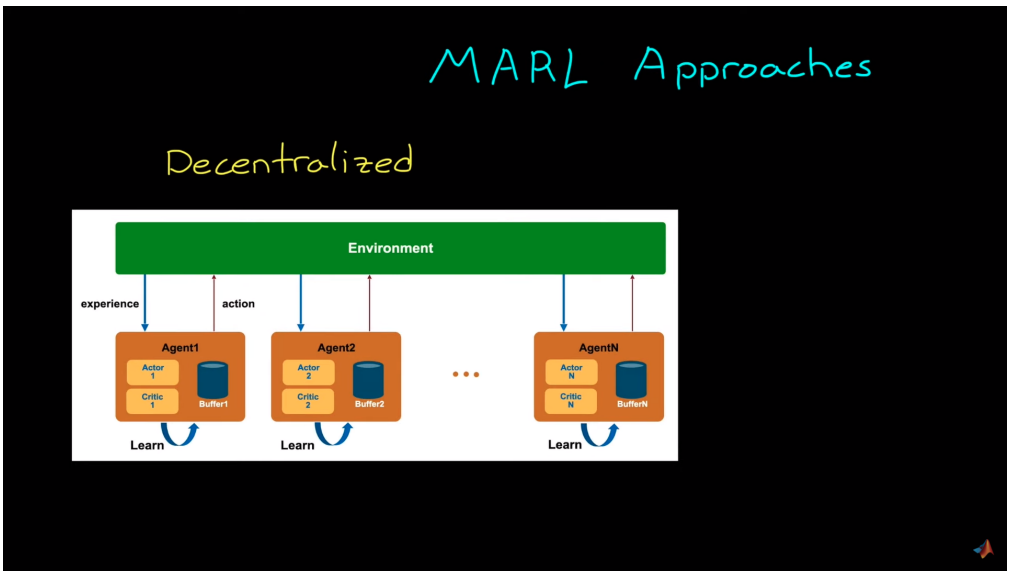


Figure 9: Decentralized multi-agent training of vacuum cleaners (Brian Douglas: Introduction to Multi-Agent Reinforcement Learning) [original figure]

In decentralized learning, **each agent is trained independently from the others**. In the example given, each vacuum learns to clean as many places as it can **without caring about what other vacuums (agents) are doing**.

The benefit is that **since no information is shared between agents, these vacuums can be designed and trained like we train single agents**.

The idea here is that **our training agent will consider other agents as part of the environment dynamics**. Not as agents.

However, the big drawback of this technique is that it will **make the environment non-stationary** since the underlying Markov decision process changes over time as other agents are also interacting in the environment. And this is problematic for many Reinforcement Learning algorithms **that can't reach a global optimum with a non-stationary environment**.

3.2 Centralized approach

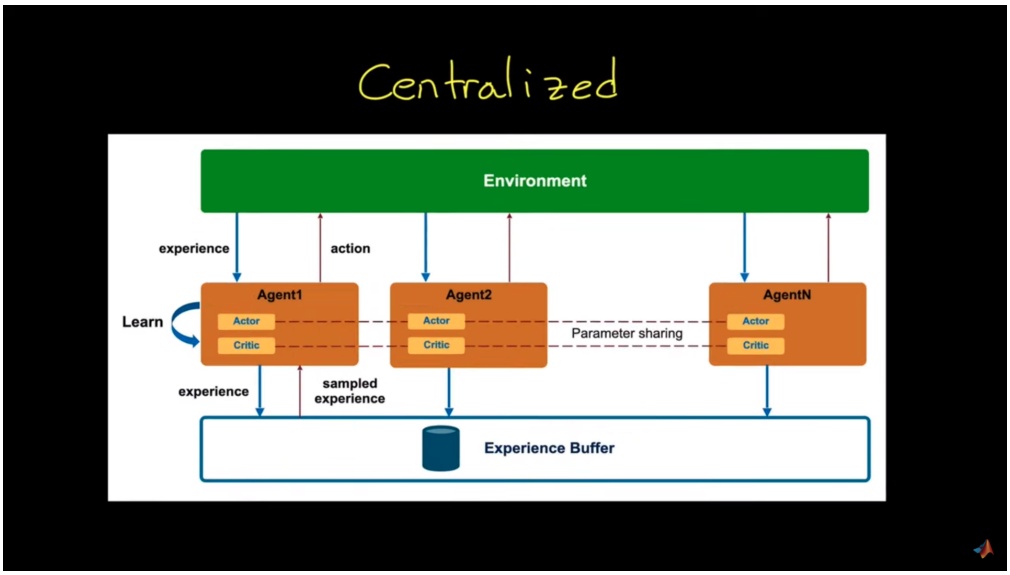


Figure 10: Centralized multi-agent training using shared experience (Brian Douglas: Introduction to Multi-Agent Reinforcement Learning) [original figure]

In this architecture, **we have a high-level process that collects agents' experiences**: the experience buffer. And we'll use these experiences **to learn a common policy**.

For instance, in the vacuum cleaner example, the observation will be:

- The coverage map of the vacuums.
- The position of all the vacuums.

We use that collective experience **to train a policy that will move all three robots in the most beneficial way as a whole**. So each robot is learning from their common experience. We now have a stationary environment since all the agents are treated as a larger entity, and they know the change of other agents' policies (since it's the same as theirs).

If we recap:

- In a *decentralized approach*, we **treat all agents independently without considering the existence of the other agents.**
  - In this case, all agents **consider others agents as part of the environment**.
  - **It's a non-stationarity environment condition**, so has no guarantee of convergence.
- In a *centralized approach*:
  - A **single policy is learned from all the agents**.
  - Takes as input the present state of an environment and the policy outputs joint actions.
  - The reward is global.

4 Self-Play: a classic technique to train competitive agents in adversarial games

Now that we've studied the basics of multi-agents, we're ready to go deeper. As mentioned in the introduction, we're going **to train agents in an adversarial game with SoccerTwos, a 2vs2 game**.

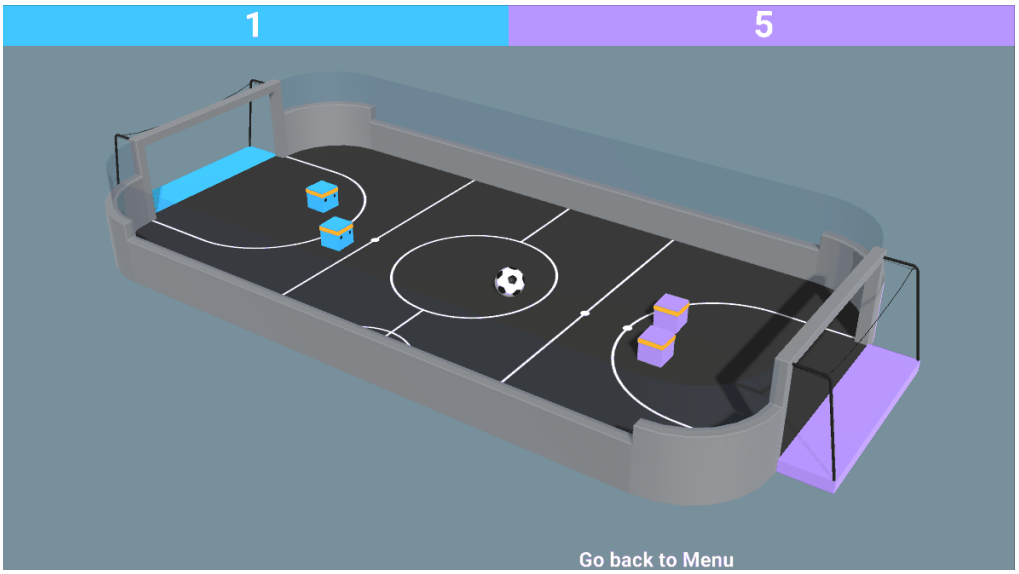


Figure 11: SoccerTwos: Unity ML-Agents two-versus-two soccer environment (Unity ML-Agents Team) [original animation]

4.1 What is Self-Play?

Training agents correctly in an adversarial game can be **quite complex**.

On the one hand, we need to find how to get a well-trained opponent to play against your training agent. And on the other hand, if you find a very good trained opponent, how will your agent improve its policy when the opponent is too strong?

Think of a child that just started to learn soccer. Playing against a very good soccer player will be useless since it will be too hard to win or at least get the ball from time to time. So the child will continuously lose without having time to learn a good policy.

The best solution would be **to have an opponent that is on the same level as the agent and will upgrade its level as the agent upgrades its own**. Because if the opponent is too strong, we'll learn nothing; if it is too weak, we'll overlearn useless behavior against a stronger opponent then.

This solution is called *self-play*. In self-play, **the agent uses former copies of itself (of its policy) as an opponent**. This way, the agent will play against an agent of the same level (challenging but not too much), have opportunities to gradually improve its policy, and then update its opponent as it becomes better. It's a way to bootstrap an opponent and progressively increase the opponent's complexity.

It's the same way humans learn in competition:

- We start to train against an opponent of similar level
- Then we learn from it, and when we acquire some skills, we can move further with stronger opponents.

We do the same with self-play:

- We **start with a copy of our agent as an opponent** this way, this opponent is on a similar level.
- We **learn from it** and, when we acquire some skills, we **update our opponent with a more recent copy of our training policy**.

The theory behind self-play is not something new. It was already used by Arthur Samuel's checker player system in the fifties and by Gerald Tesauro's TD-Gammon in 1995. If you want to learn more about the history of self-play check out this very good blogpost by Andrew Cohen

4.2 Self-Play in MLAgents

Self-Play is integrated into the MLAgents library and is managed by multiple hyperparameters that we're going to study. But the main focus, as explained in the documentation, is the **tradeoff between the skill level and generality of the final policy and the stability of learning**.

Training against a set of slowly changing or unchanging adversaries with low diversity **results in more stable training. But there is a risk of overfitting if the change is too slow.**

So we need to control:

- How **often we change opponents** with the `swap_steps` and `team_change` parameters.
- The **number of opponents saved** with the `window` parameter. A larger value of `window` means that an agent's pool of opponents will contain a larger diversity of behaviors since it will contain policies from earlier in the training run.
- The **probability of playing against the current self vs opponent** sampled from the pool with `play_against_latest_model_ratio`. A larger value of `play_against_latest_model_ratio` indicates that an agent will be playing against the current opponent more often.
- The **number of training steps before saving a new opponent** with `save_steps` parameters. A larger value of `save_steps` will yield a set of opponents that cover a wider range of skill levels and possibly play styles since the policy receives more training.

To get more details about these hyperparameters, you definitely need to check out this part of the documentation

4.3 The ELO Score to evaluate our agent

What is ELO Score?

In adversarial games, tracking the **cumulative reward is not always a meaningful metric to track the learning progress:** because this metric is **dependent only on the skill of the opponent.**

Instead, we're using an ***ELO rating system*** (named after Arpad Elo) that calculates the **relative skill level** between 2 players from a given population in a zero-sum game.

In a zero-sum game: one agent wins, and the other agent loses. It's a mathematical representation of a situation in which each participant's gain or loss of utility **is exactly balanced by the gain or loss of the utility of the other participants.** We talk about zero-sum games because the sum of utility is equal to zero.

This ELO (starting at a specific score: frequently 1200) can decrease initially but should increase progressively during the training.

The Elo system is **inferred from the losses and draws against other players.** It means that player ratings depend **on the ratings of their opponents and the results scored against them.**

Elo defines an Elo score that is the relative skills of a player in a zero-sum game. **We say relative because it depends on the performance of opponents.**

The central idea is to think of the performance of a player **as a random variable that is normally distributed.**

The difference in rating between 2 players serves as **the predictor of the outcomes of a match.** If the player wins, but the probability of winning is high, it will only win a few points from its opponent since it means that it is much stronger than it.

After every game:

- The winning player takes **points from the losing one.**

- The number of points is determined **by the difference in the 2 players ratings (hence relative).**
  - If the higher-rated player wins → few points will be taken from the lower-rated player.
  - If the lower-rated player wins → a lot of points will be taken from the high-rated player.
  - If it's a draw → the lower-rated player gains a few points from the higher.

So if A and B have rating Ra, and Rb, then the **expected scores are** given by:

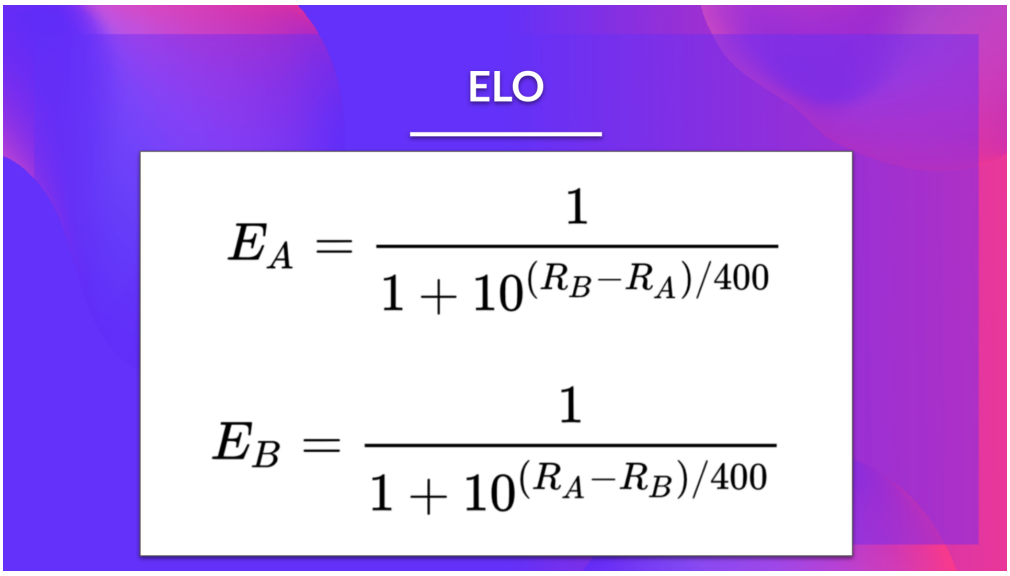


Figure 12: Elo expected-score formula for two players [original figure]

Then, at the end of the game, we need to update the player's actual Elo score. We use a linear adjustment **proportional to the amount by which the player over-performed or under-performed.**

We also define a maximum adjustment rating per game: K-factor.

- K=16 for master.
- K=32 for weaker players.

If Player A has Ea points but scored Sa points, then the player's rating is updated using the formula:

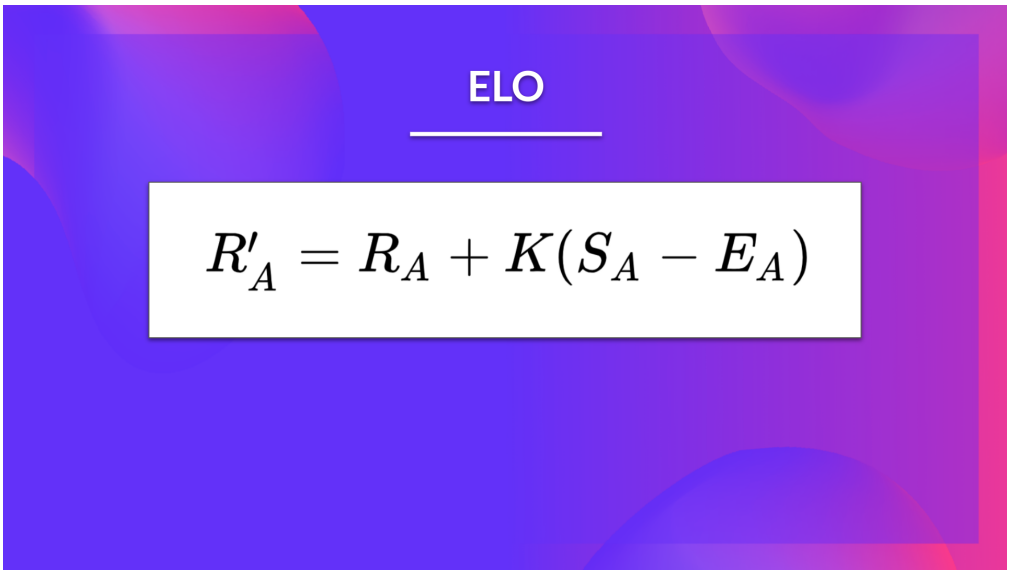


Figure 13: Elo rating-update formula after a match [original figure]

Example

If we take an example:

Player A has a rating of 2600

Player B has a rating of 2300

- We first calculate the expected score:

$E_A = \frac{1}{1+10^{(2300-2600)/400}} = 0.849$

$E_B = \frac{1}{1+10^{(2600-2300)/400}} = 0.151$

- If the organizers determined that K=16 and A wins, the new rating would be:

$ELO_A = 2600 + 16 * (1 - 0.849) = 2602$

$ELO_B = 2300 + 16 * (0 - 0.151) = 2298$

- If the organizers determined that K=16 and B wins, the new rating would be:

$ELO_A = 2600 + 16 * (0 - 0.849) = 2586$

$ELO_B = 2300 + 16 * (1 - 0.151) = 2314$

The Advantages

Using the ELO score has multiple advantages:

- Points are **always balanced** (more points are exchanged when there is an unexpected outcome, but the sum is always the same).
- It is a **self-corrected system** since if a player wins against a weak player, they will only win a few points.
- It **works with team games**: we calculate the average for each team and use it in Elo.

The Disadvantages

- ELO **does not take into account the individual contribution** of each people in the team.
- Rating deflation: **a good rating requires skill over time to keep the same rating**.
- **Can't compare rating in history**.

5 Hands-on

Now that you learned the basics of multi-agents, you're ready to train your first agents in a multi-agent system: **a 2vs2 soccer team that needs to beat the opponent team**.

And you're going to participate in AI vs. AI challenges where your trained agent will compete against other classmates' **agents every day and be ranked on a new leaderboard.**

To validate this hands-on for the certification process, you just need to push a trained model. There **are no minimal results to attain to validate it.**

For more information about the certification process, check this section https://huggingface.co/deep-rl-course/en/unit0/introduction#certification-process

This hands-on will be different since to get correct results **you need to train your agents from 4 hours to 8 hours**. And given the risk of timeout in Colab, we advise you to train on your computer. You don't need a supercomputer: a simple laptop is good enough for this exercise.

Let's get started!

5.1 What is AI vs. AI?

AI vs. AI is an open-source tool we developed at Hugging Face to compete agents on the Hub against one another in a multi-agent setting. These models are then ranked in a leaderboard.

The idea of this tool is to have a robust evaluation tool: **by evaluating your agent with a lot of others, you'll get a good idea of the quality of your policy.**

More precisely, AI vs. AI is three tools:

- A *matchmaking process* defining the matches (which model against which) and running the model fights using a background task in the Space.
- A *leaderboard* getting the match history results and displaying the models' ELO ratings: https://huggingface.co/spaces/huggingface-projects/AIvsAI-SoccerTwos
- A *Space demo* to visualize your agents playing against others: https://huggingface.co/spaces/unity/ML-Agents-SoccerTwos

In addition to these three tools, your classmate cyllum created a SoccerTwos Challenge Analytics where you can check the detailed match results of a model: https://huggingface.co/spaces/cyllum/soccertwos-analytics

We're wrote a blog post to explain this AI vs. AI tool in detail, but to give you the big picture it works this way:

- Every four hours, our algorithm **fetches all the available models for a given environment (in our case ML-Agents-SoccerTwos).**
- It creates a **queue of matches with the matchmaking algorithm.**
- We simulate the match in a Unity headless process and **gather the match result** (1 if the first model won, 0.5 if it's a draw, 0 if the second model won) in a Dataset.
- Then, when all matches from the matches queue are done, **we update the ELO score for each model and update the leaderboard.**

Competition Rules

This first AI vs. AI competition **is an experiment**: the goal is to improve the tool in the future with your feedback. So some **breakups can happen during the challenge**. But don't worry **all the results are saved in a dataset so we can always restart the calculation correctly without losing information**.

In order for your model to get correctly evaluated against others you need to follow these rules:

1. **You can't change the observation space or action space of the agent.** By doing that your model will not work during evaluation.
2. You **can't use a custom trainer for now,** you need to use the Unity MLAgents ones.
3. We provide executables to train your agents. You can also use the Unity Editor if you prefer **, but to avoid bugs, we advise that you use our executables**.

What will make the difference during this challenge are **the hyperparameters you choose**.

We're constantly trying to improve our tutorials, so **if you find some issues in this notebook**, please open an issue on the GitHub Repo.

Chat with your classmates, share advice and ask questions on Discord

- We created a new channel called `ai-vs-ai-challenge` to exchange advice and ask questions.
- If you didn't join the discord server yet, you can join here

5.2 Step 0: Install MLAgents and download the correct executable

We advise you to use conda as a package manager and create a new environment.

With conda, we create a new environment called rl with **Python 3.10.12**:

```
conda create --name rl python=3.10.12
conda activate rl
```

To be able to train our agents correctly and push to the Hub, we need to install ML-Agents

```
git clone https://github.com/Unity-Technologies/ml-agents
```

When the cloning is done (it takes 2.63 GB), we go inside the repository and install the package

```
cd ml-agents
pip install -e ./ml-agents-envs
pip install -e ./ml-agents
```

Mac users on Apple Silicon may encounter troubles with the installation (e.g. ONNX wheel build failing), you should first try to install grpcio:

```
conda install grpcio
```

This github issue in the official ml-agent repo might also help you.

Finally, you need to install git-lfs: https://git-lfs.com/

Now that it's installed, we need to add the environment training executable. Based on your operating system you need to download one of them, unzip it and place it in a new folder inside `ml-agents` that you call `training-envs-executables`

At the end your executable should be in `ml-agents/training-envs-executables/SoccerTwos`

Windows: Download this executable

Linux (Ubuntu): Download this executable

Mac: Download this executable Warning: For Mac you need also to call this `xattr -cr training-envs-executables/SoccerTwos/SoccerTwos.app` to be able to run SoccerTwos

5.3 Step 1: Understand the environment

The environment is called `SoccerTwos`. The Unity MLAgents Team made it. You can find its documentation here

The goal in this environment **is to get the ball into the opponent's goal while preventing the ball from entering your own goal.**

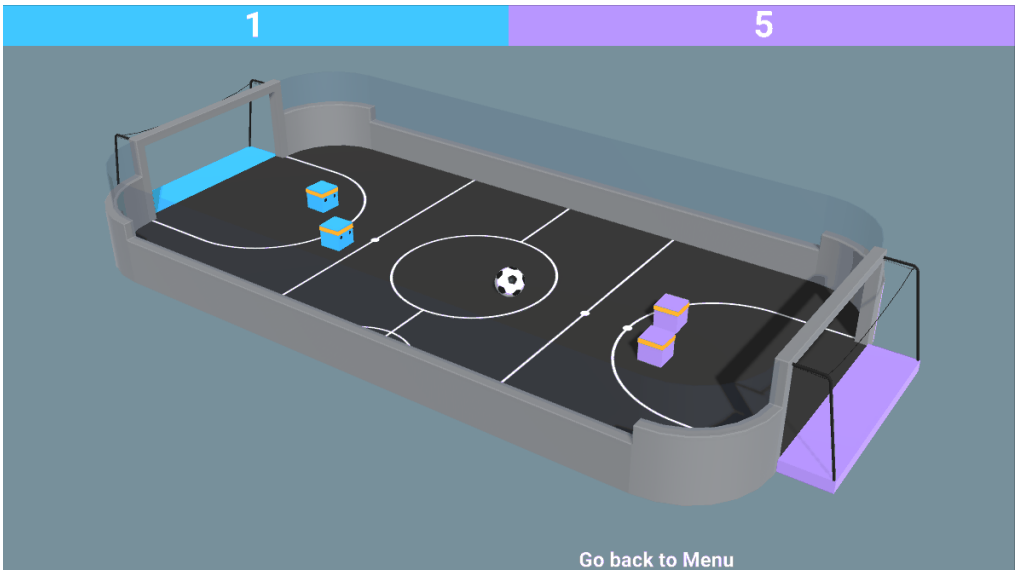


Figure 14: SoccerTwos: Unity ML-Agents two-versus-two soccer environment (Unity ML-Agents Team) [original animation]

The reward function

The reward function is:

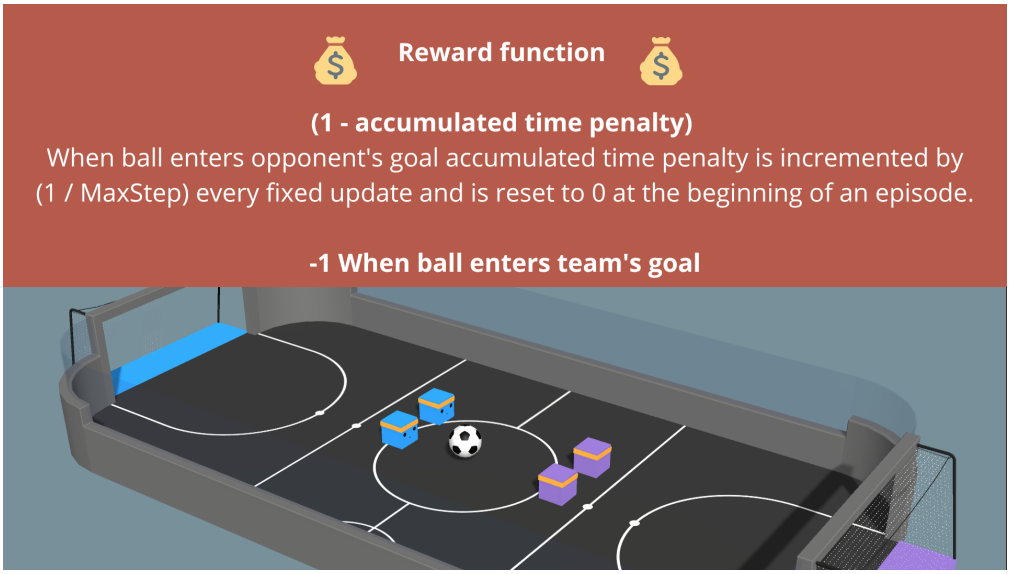


Figure 15: SoccerTwos: the environment reward function [original figure]

The observation space

The observation space is composed of vectors of size 336:

- 11 ray-casts forward distributed over 120 degrees (264 state dimensions)
- 3 ray-casts backward distributed over 90 degrees (72 state dimensions)
- Both of these ray-casts can detect 6 objects:
  - Ball
  - Blue Goal
  - Purple Goal
  - Wall
  - Blue Agent
  - Purple Agent

The action space

The action space is three discrete branches:



Figure 16: SoccerTwos: discrete and continuous action spaces [original figure]

5.4 Step 2: Understand MA-POCA

We know how to train agents to play against others: **we can use self-play.** This is a perfect technique for a 1vs1.

But in our case we're 2vs2, and each team has 2 agents. How then can we **train cooperative behavior for groups of agents?**

As explained in the Unity Blog, agents typically receive a reward as a group (+1 - penalty) when the team scores a goal. This implies that **every agent on the team is rewarded even if each agent didn't contribute the same to the win**, which makes it difficult to learn what to do independently.

The Unity MLAgents team developed the solution in a new multi-agent trainer called *MA-POCA (Multi-Agent POsthumous Credit Assignment).*

The idea is simple but powerful: a centralized critic **processes the states of all agents in the team to estimate how well each agent is doing**. Think of this critic as a coach.

This allows each agent to **make decisions based only on what it perceives locally**, and **simultaneously evaluate how good its behavior is in the context of the whole group**.

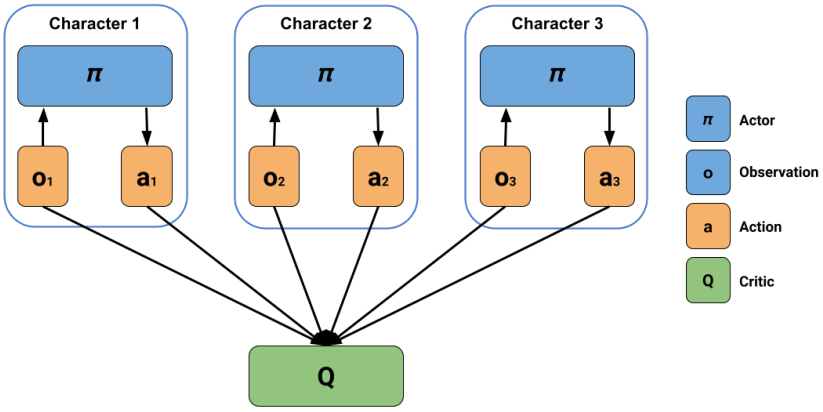


Figure 17: MA-POCA centralized learning and decentralized execution (source: ML-Agents Plays Dodgeball) [original figure]

The solution then is to use Self-Play with an MA-POCA trainer (called poca). The poca trainer will help us to train cooperative behavior and self-play to win against an opponent team.

If you want to dive deeper into this MA-POCA algorithm, you need to read the paper they published here and the sources we put on the additional readings section.

5.5 Step 3: Define the config file

We already learned in Unit 5 that in ML-Agents, you define **the training hyperparameters in `config.yaml` files.**

There are multiple hyperparameters. To understand them better, you should read the explanations for each of them in **the documentation**

The config file we're going to use here is in `./config/poca/SoccerTwos.yaml`. It looks like this:

```
behaviors:
  SoccerTwos:
    trainer_type: poca
    hyperparameters:
      batch_size: 2048
      buffer_size: 20480
      learning_rate: 0.0003
      beta: 0.005
      epsilon: 0.2
      lambd: 0.95
      num_epoch: 3
      learning_rate_schedule: constant
    network_settings:
      normalize: false
      hidden_units: 512
      num_layers: 2
      vis_encode_type: simple
    reward_signals:
      extrinsic:
        gamma: 0.99
        strength: 1.0
    keep_checkpoints: 5
    max_steps: 5000000
    time_horizon: 1000
    summary_freq: 10000
    self_play:
```

```
      save_steps: 50000
      team_change: 200000
      swap_steps: 2000
      window: 10
      play_against_latest_model_ratio: 0.5
      initial_elo: 1200.0
```

Compared to Pyramids or SnowballTarget, we have new hyperparameters with a self-play part. How you modify them can be critical in getting good results.

The advice I can give you here is to check the explanation and recommended value for each parameters (especially self-play ones) against **the documentation.**

Now that you've modified our config file, you're ready to train your agents.

5.6 Step 4: Start the training

To train the agents, we need to **launch mlagents-learn and select the executable containing the environment.**

We define four parameters:

1. `mlagents-learn <config>`: the path where the hyperparameter config file is.
2. `-env`: where the environment executable is.
3. `-run_id`: the name you want to give to your training run id.
4. `-no-graphics`: to not launch the visualization during the training.

Depending on your hardware, 5M timesteps (the recommended value, but you can also try 10M) will take 5 to 8 hours of training. You can continue using your computer in the meantime, but I advise deactivating the computer standby mode to prevent the training from being stopped.

Depending on the executable you use (windows, ubuntu, mac) the training command will look like this (your executable path can be different so don't hesitate to check before running).

For Windows, it might look like this:

```
mlagents-learn ./config/poca/SoccerTwos.yaml --env=./training-envs-executables/SoccerTwos.exe --run-
    id="SoccerTwos" --no-graphics
```

For Mac, it might look like this:

```
mlagents-learn ./config/poca/SoccerTwos.yaml --env=./training-envs-executables/SoccerTwos/
    SoccerTwos.app --run-id="SoccerTwos" --no-graphics
```

The executable contains 8 copies of SoccerTwos.

Warning: It's normal if you don't see a big increase of ELO score (and even a decrease below 1200) before 2M timesteps, since your agents will spend most of their time moving randomly on the field before being able to goal.

Warning: You can stop the training with Ctrl + C but beware of typing this command only once to stop the training since MLAgents needs to generate a final .onnx file before closing the run.

5.7 Step 5: Push the agent to the Hugging Face Hub

Now that we trained our agents, we're **ready to push them to the Hub to be able to participate in the AI vs. AI challenge and visualize them playing on your browser.**

To be able to share your model with the community, there are three more steps to follow:

1. (If it's not already done) create an account to HF -> https://huggingface.co/join

2. Sign in and store your authentication token from the Hugging Face website.

Create a new token (https://huggingface.co/settings/tokens) **with write role**

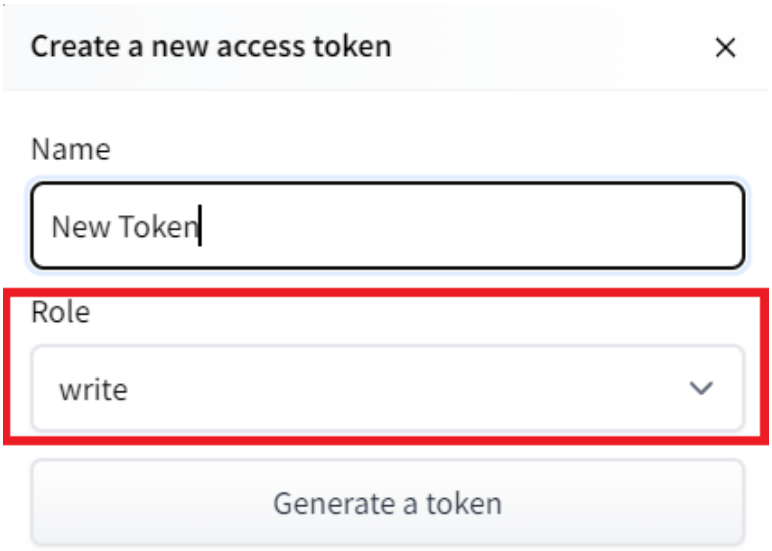


Figure 18: Creating a Hugging Face write-access token [original figure]

Copy the token, run this, and paste the token

```
huggingface-cli login
```

Then, we need to run `mlagents-push-to-hf`.

And we define four parameters:

1. `-run-id`: the name of the training run id.
2. `-local-dir`: where the agent was saved, it's results/, so in my case results/First Training.
3. `-repo-id`: the name of the Hugging Face repo you want to create or update. It's always `<your huggingface username>/<the repo name>` If the repo does not exist **it will be created automatically**
4. `--commit-message`: since HF repos are git repositories you need to give a commit message.

In my case

```
mlagents-push-to-hf  --run-id="SoccerTwos" --local-dir="./results/SoccerTwos" --repo-id="
    ThomasSimonini/poca-SoccerTwos" --commit-message="First Push"`
```

```
mlagents-push-to-hf  --run-id= # Add your run id  --local-dir= # Your local dir  --repo-id= # Your
    repo id --commit-message="First Push"
```

If everything worked you should see this at the end of the process (but with a different url) :

Your model is pushed to the Hub. You can view your model here: https://huggingface.co/ThomasSimonini/poca-SoccerTwos

It's the link to your model. It contains a model card that explains how to use it, your Tensorboard, and your config file. **What's awesome is that it's a git repository, which means you can have different commits, update your repository with a new push, etc.**

5.8 Step 6: Verify that your model is ready for AI vs AI Challenge

Now that your model is pushed to the Hub, **it's going to be added automatically to the AI vs AI Challenge model pool.** It can take a little bit of time before your model is added to the leaderboard given we do a run of matches every 4h.

But to ensure that everything works perfectly you need to check:

1. That you have this tag in your model: ML-Agents-SoccerTwos. This is the tag we use to select models to be added to the challenge pool. To do that go to your model and check the tags

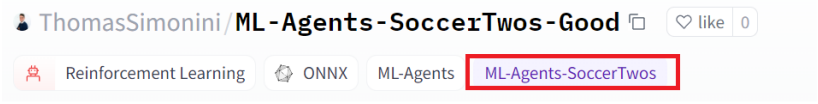


Figure 19: Checking the ML-Agents-SoccerTwos model tag [original figure]

If it's not the case you just need to modify the readme and add it

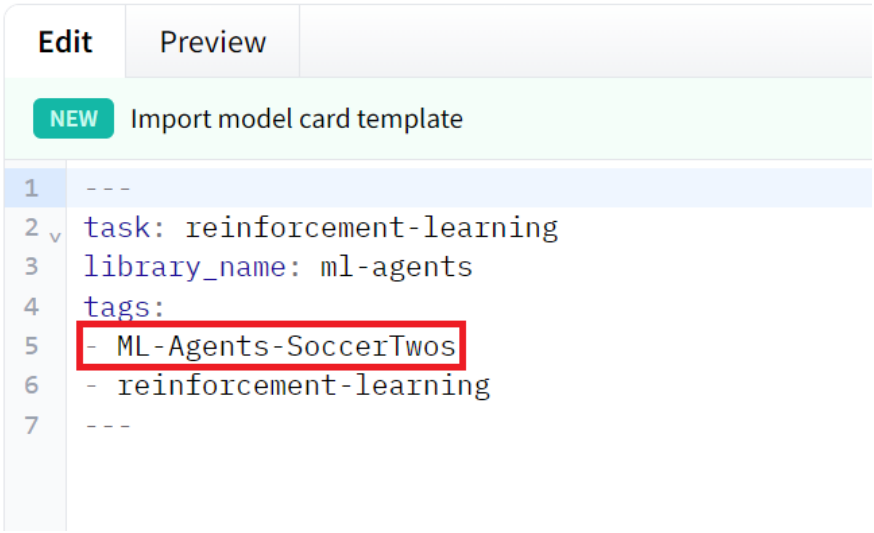


Figure 20: Adding the SoccerTwos tag in the Model Card [original figure]

2. That you have a `SoccerTwos.onnx` file

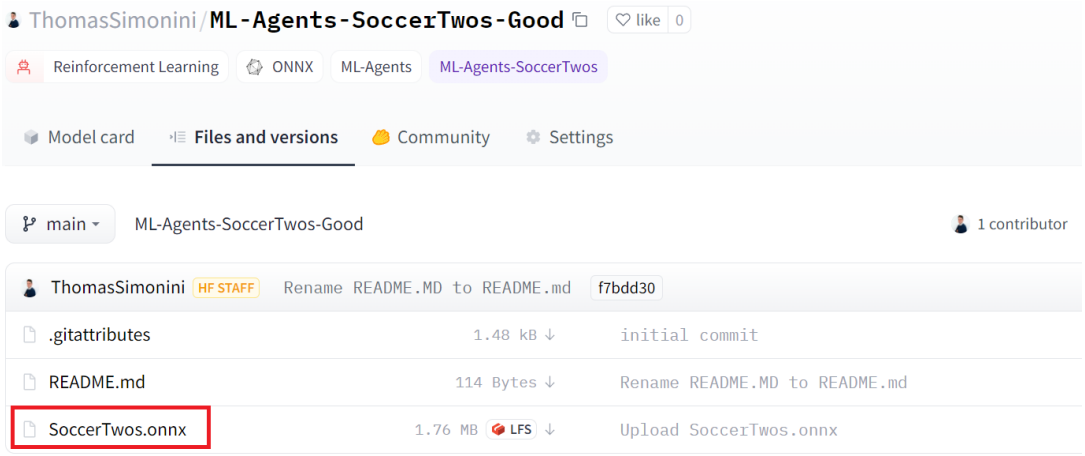


Figure 21: Checking that SoccerTwos.onnx was uploaded [original figure]

We strongly suggest that you create a new model when you push to the Hub if you want to train it again or train a new version.

5.9 Step 7: Visualize some match in our demo

Now that your model is part of AI vs AI Challenge, **you can visualize how good it is compared to others**: https://huggingface.co/spaces/unity/ML-Agents-SoccerTwos

In order to do that, you just need to go to this demo:

- Select your model as team blue (or team purple if you prefer) and another model to compete against. The best opponents to compare your model to are either whoever is on top of the leaderboard or the baseline model

The matches you see live are not used in the calculation of your result **but they are a good way to visualize how good your agent is**.

And don't hesitate to share the best score your agent gets on discord in the #rl-i-made-this channel

6 Quiz

The best way to learn and to avoid the illusion of competence **is to test yourself.** This will help you to find **where you need to reinforce your knowledge**.

6.1 Q1: Chose the option which fits better when comparing different types of multi-agent environments

- Your agents aim to maximize common benefits in _____ environments
- Your agents aim to maximize common benefits while minimizing opponent's in _____ environments
- **A.** competitive, cooperative
- **B.** cooperative, competitive

6.2 Q2: Which of the following statements are true about `decentralized` learning?

- **A.** Each agent is trained independently from the others
- **B.** Inputs from other agents are just considered environment data
- **C.** Considering other agents part of the environment makes the environment stationary

6.3 Q3: Which of the following statements are true about `centralized` learning?

- **A.** It learns one common policy based on the learnings from all agents' interactions
- **B.** The reward is global
- **C.** The environment with this approach is stationary

6.4 Q4: Explain in your own words what is the `Self-Play` approach

6.5 Q5: When configuring `Self-play`, several parameters are important. Could you identify, by their definition, which parameter are we talking about?

- The probability of playing against the current self vs an opponent from a pool
- Variety (dispersion) of training levels of the opponents you can face
- The number of training steps before spawning a new opponent
- Opponent change rate
- **A.** window, play_against_latest_model_ratio, save_steps, swap_steps+team_change
- **B.** play_against_latest_model_ratio, save_steps, window, swap_steps+team_change
- **C.** play_against_latest_model_ratio, window, save_steps, swap_steps+team_change
- **D.** swap_steps+team_change, save_steps, play_against_latest_model_ratio, window

6.6 Q6: What are the main motivations to use a ELO rating Score?

- **A.** The score takes into account the different of skills between you and your opponent
- **B.** Although more points can be exchanged depending on the result of the match and given the levels of the agents, the sum is always the same
- **C.** It's easy for an agent to keep a high score rate
- **D.** It works well calculating the individual contributions of each player in a team

Congrats on finishing this Quiz , if you missed some elements, take time to read the chapter again to reinforce () your knowledge.

6.7 Quiz solutions and explanations

Q1 solution **Correct answer(s):** B

A. competitive, cooperative Explanation: You maximize common benefit in cooperative, while in competitive you also aim to reduce opponent's score

B. cooperative, competitive

Q2 solution **Correct answer(s):** A, B

A. Each agent is trained independently from the others

B. Inputs from other agents are just considered environment data

C. Considering other agents part of the environment makes the environment stationary Explanation: In decentralized learning, agents ignore the existence of other agents and consider them part of the environment. However, this means the environment is in constant change, becoming non-stationary.

Q3 solution **Correct answer(s):** A, B, C

A. It learns one common policy based on the learnings from all agents' interactions

B. The reward is global

C. The environment with this approach is stationary

Q4 solution `Self-play` is an approach to instantiate copies of agents with the same policy as your as opponents, so that your agent learns from agents with same training level.

Q5 solution **Correct answer(s):** C

A. window, play_against_latest_model_ratio, save_steps, swap_steps+team_change

B. play_against_latest_model_ratio, save_steps, window, swap_steps+team_change

C. play_against_latest_model_ratio, window, save_steps, swap_steps+team_change

D. swap_steps+team_change, save_steps, play_against_latest_model_ratio, window

Q6 solution **Correct answer(s):** A, B

A. The score takes into account the different of skills between you and your opponent

B. Although more points can be exchanged depending on the result of the match and given the levels of the agents, the sum is always the same

C. It's easy for an agent to keep a high score rate Explanation: That is called the `Rating deflation`: keeping a high rate requires much skill over time

D. It works well calculating the individual contributions of each player in a team Explanation: ELO uses the score achieved by the whole team, but individual contributions are not calculated

7 Conclusion

That's all for today. Congrats on finishing this unit and the tutorial!

The best way to learn is to practice and try stuff. **Why not train another agent with a different configuration?**

And don't hesitate from time to time to check the leaderboard

See you in Unit 8

7.1 Keep Learning, Stay awesome

8 Additional Readings

8.1 An introduction to multi-agents

- Multi-agent reinforcement learning: An overview
- Multiagent Reinforcement Learning, Marc Lanctot
- Example of a multi-agent environment
- A list of different multi-agent environments
- Multi-Agent Reinforcement Learning: Independent vs. Cooperative Agents
- Dealing with Non-Stationarity in Multi-Agent Deep Reinforcement Learning

8.2 Self-Play and MA-POCA

- Self Play Theory and with MLAgents
- Training complex behavior with MLAgents
- MLAgents plays dodgeball
- On the Use and Misuse of Absorbing States in Multi-agent Reinforcement Learning (MA-POCA)